الهيئة الاتحادية للهوية
والجنسية والجمارك وأمن المنافذ

FEDERAL AUTHORITY FOR IDENTITY,
CITIZENSHIP, CUSTOMS & PORT SECURITY

EIDA Toolkit iOS Integration Guide

Version 1.0 -- ID Card Toolkit SDK v3.1.6

May 2026

CONFIDENTIAL

Federal Authority for Identity, Citizenship, Customs & Port Security (ICP)
United Arab Emirates

Table of Contents

Revision History

Version	Date	Description of Changes
1.0	May 2026	Initial release. Covers ICASwiftApp (regular Swift sample) and IDCardToolkitObjCApp (Objective-C sample) for SDK v3.1.6.

1. Introduction

This document is for an iOS developer who needs to:

- Build and run the bundled reference applications that ship with the ID Card Toolkit iOS SDK.
- Integrate the toolkit into a new iOS application.

Two reference applications ship with the SDK and are both covered here:

- **ICASwiftApp** — the regular Swift sample (Section 4).
- **IDCardToolkitObjCApp** — the Objective-C sample (Section 5).

Both samples demonstrate the same toolkit API surface: PC/SC reader connection, public-data read, NFC read, PKI authentication, biometric capture, and digital-signature operations (CAdES / XAdES / PAdES). Pick the sample whose host language matches your codebase; the integration steps for a brand-new app (Section 6) are language-agnostic.

After working through this guide an iOS developer should be able to compile, sign, and run either sample on an iPhone or iPad, and adapt the same integration pattern to their own application.

2. Prerequisites

2.1. Hardware

Item	Requirement
macOS host	Apple Silicon or Intel; macOS 12 or newer
iOS test device	iPhone or iPad with NFC reader (iPhone 7 or later)
Smart-card reader	Any of the supported readers (e.g. PreciseBiometrics Tactivo, FTSafe iR301, Feitian iR301) — required for contact-mode card reading

2.2. Software

Item	Requirement
Xcode	13.0 or newer (Xcode 15+ recommended for current iOS SDK)
CocoaPods	1.10+ (required for the Swift sample's pod dependencies)
Apple Developer account	Required to sign the app for on-device testing. A free personal account works for local-device runs but does not provision NFC.
ID Card Toolkit iOS SDK	`id-card-toolkit-ios-sdk-v3.1.6` (this distribution)
Configuration files	Licensed configuration bundle for your environment (test or production), issued by ICP

3. SDK Layout

Extract the SDK zip. The directory layout is as follows:

```
id-card-toolkit-ios-sdk-v3.1.6/
├── README
├── VERSION
├── doc/                                Developer PDFs
├── lib/
│   ├── objective-c/IDCardToolkit.framework
│   └── swift/IDCardToolkit.framework
├── plugins/
│   ├── Feitian/Feitian.framework
│   ├── Grabba/Grabba.framework
│   ├── NFC/iOSNfcPlugin.framework
│   ├── Tactivo/libTactivo.framework
│   └── Tech5/                          6 frameworks shipped as siblings:
│       ├── Tech5AirSnapFinger.framework      ICP toolkit plugin wrapper
│       ├── AirSnapFingerUI.framework         Tech5 vendor
│       ├── T5AirSnapFramework.framework      Tech5 vendor
│       ├── ncnn.framework                    Tech5 vendor (ML runtime)
│       ├── opencv2.framework                 Tech5 vendor (image processing)
│       └── OpenSSL.framework                 Tech5 vendor dependency
├── ext/
│   ├── openssl/include/iOS/             OpenSSL headers
│   ├── openssl/lib/iOS/                 libssl.a, libcrypto.a (device + simulator)
│   ├── xmlsec/include/MacOSX/           xmlsec + libxml headers
│   └── xmlsec/lib/iOS/                  libxmlsec.a, libxmlsec1-openssl.a
└── samples/
    ├── ICASwiftApp/                     Regular Swift reference app (Section 4)
    └── IDCardToolkitObjCApp/            Objective-C reference app (Section 5)
```

Frameworks under `lib/` are the toolkit's two API surfaces (Objective-C and Swift). Frameworks under `plugins/` are vendor reader / capture plugins — embed only the ones whose hardware your app supports. Files under `ext/` are sample-only third-party headers and static archives that the bundled samples link against directly; a minimal application that only calls EIDA Toolkit APIs does not need them.

4. Build and Run the Swift Sample (ICASwiftApp)

The fastest way to verify the SDK works on your device is to build and run the bundled Swift sample. All paths in this section are relative to the SDK root. If your codebase is Objective-C, see Section 5 for the ObjC sample walkthrough instead.

4.1. Install CocoaPods dependencies

The sample uses two CocoaPods (`QKMRZParser`, `SwiftyTesseract`). From a terminal:

```
cd samples/ICASwiftApp
pod install
```

> **Note:** If `pod` is not installed, run `sudo gem install cocoapods` first. Once `pod install` completes you must open the `.xcworkspace` file (not the `.xcodeproj`) so the Pods project is built alongside the app.

4.2. Open the project in Xcode

```
open samples/ICASwiftApp/ICASwiftApp.xcworkspace
```

4.3. Set your signing identity

1. Select the `ICASwiftApp` target in the project navigator.
2. Open the **Signing & Capabilities** tab.
3. Change the Bundle Identifier from `ae.emiratesid.idcard.toolkit.sample` to a unique identifier you own (e.g. `com.yourcompany.icasample`).
4. Under **Team**, select your own Apple Developer team. The bundled value is ICP's team identifier and will not work for you.
5. Confirm the Provisioning Profile area shows "Xcode Managed Profile" with no red warnings.

> **Note:** The sample target already declares the **Near Field Communication Tag Reading** capability in its entitlements. Your selected team must be entitled to use NFC on iOS; this is automatic for paid Developer Program teams and is not available for free personal teams.

4.4. Deploy configuration files

Place the configuration files issued by ICP into the project's Resources group so they ship inside the app bundle. `AppDelegate` copies them out to the Documents directory on first launch:

```
// AppDelegate.swift
self.copyFilesToDocumentDirectory("config_li",       subFileName: "config_li")
self.copyFilesToDocumentDirectory("config_pg",       subFileName: "config_pg")
self.copyFilesToDocumentDirectory("config_lv_prod",  subFileName: "config_lv_prod")
self.copyFilesToDocumentDirectory("config_tk_prod",  subFileName: "config_tk_prod")
self.copyFilesToDocumentDirectory("config_vg_prod",  subFileName: "config_vg_prod")
```

Required configuration files for a typical production deployment:

File	Purpose	Issued by
`config_li`	License file (per-deployment)	ICP
`config_pg`	Plugin manifest	ICP
`config_tk_prod`	Toolkit policy	ICP
`config_vg_prod`	Validation Gateway endpoints	ICP
`config_lv_prod`	License validation	ICP

> **Note: QA / Test environment.** For test builds, swap `config_tk_prod` / `config_vg_prod` / `config_lv_prod` with the matching `_qa` variants and update the `AppDelegate` copy calls accordingly.

4.5. Build and run

1. Choose your connected iPhone or iPad from the destination dropdown in Xcode.
2. Press Cmd-R (**Product > Run**).
3. On first launch on a device, iOS will refuse to open the app until you trust the developer certificate. Open **Settings > General > VPN & Device Management** on the device and trust your team's profile.
4. The sample opens its main screen with a list of feature areas (read public data, NFC, PKI, signing, etc.). Without a smart-card reader attached you can still exercise the NFC-based reads against a real Emirates ID card.

5. Build and Run the Objective-C Sample (IDCardToolkitObjCApp)

`IDCardToolkitObjCApp` is the Objective-C equivalent of the Swift sample covered in Section 4. It exercises the same toolkit API surface against the same configuration files; only the host language and project layout differ. Use this sample if your application code is Objective-C, or if you want a CocoaPods-free starting point.

5.1. Differences from the Swift sample

Aspect	ICASwiftApp	IDCardToolkitObjCApp
File you open in Xcode	`.xcworkspace` (Pods present)	`.xcodeproj` (no Pods)
CocoaPods setup	`pod install` required	Not used — no Podfile
Toolkit framework path	`lib/swift/IDCardToolkit.framework`	`lib/objective-c/IDCardToolkit.framework`
NFC formats in entitlements	`TAG`	`TAG`, `NDEF`
Source language	Swift 5.0	Objective-C

Everything else — plugin frameworks, Info.plist keys, external-accessory protocols, signing approach, configuration-file deployment pattern — is identical to the Swift sample.

5.2. Open the project

There is no `.xcworkspace` and no pods to install:

```
open samples/IDCardToolkitObjCApp/IDCardToolkitObjCApp.xcodeproj
```

5.3. Set your signing identity

Same procedure as for the Swift sample (Section 4.3): change the Bundle Identifier to a unique value you own, select your own Apple Developer Team under **Signing & Capabilities**, and confirm the Provisioning Profile is generated without errors.

5.4. Deploy configuration files

Add the same set of `config_*` files (Section 4.4) to the project's Resources group. The AppDelegate already copies them to the Documents directory on first launch — the Objective-C equivalent of the Swift snippet:

```
// AppDelegate.m
[self copyFilesToDocumentDirectory:@"config_li"      subFileName:@"config_li"];
[self copyFilesToDocumentDirectory:@"config_pg"      subFileName:@"config_pg"];
[self copyFilesToDocumentDirectory:@"config_lv_prod" subFileName:@"config_lv_prod"];
[self copyFilesToDocumentDirectory:@"config_tk_prod" subFileName:@"config_tk_prod"];
[self copyFilesToDocumentDirectory:@"config_vg_prod" subFileName:@"config_vg_prod"];
```

5.5. Build and run

Identical to the Swift sample (Section 4.5): pick your iOS device, press Cmd-R, trust the developer certificate on the device on first launch, then exercise the toolkit features from the app's main menu.

5.6. Initializing the toolkit (Objective-C)

Minimal Objective-C initialization once the configuration files are deployed:

```
#import <IDCardToolkit/IDCardToolkit.h>

Toolkit *toolkit = [[Toolkit alloc] init];
NSError *error = nil;
BOOL ok = [toolkit initialize:&error];
if (!ok) {
    NSLog(@"Initialize failed: %@", error);
    return;
}
// ... call API methods (reader list, public data read, signing, etc.)
```

6. Integrating the Toolkit into Your Own App

This section covers integrating `IDCardToolkit.framework` (and the plugins you need) into a new Xcode iOS project. The steps mirror what the bundled samples do.

6.1. Add the toolkit framework

1. Drag `lib/swift/IDCardToolkit.framework` (or `lib/objective-c/IDCardToolkit.framework` if your app is pure Objective-C) into your Xcode project.
2. In the dialog choose **Create groups** (not **Create folder references**) and uncheck **Copy items if needed** if you want the project to reference the SDK location in place.
3. Open **Target > General > Frameworks, Libraries, and Embedded Content**. Confirm the framework is listed and set **Embed = Embed & Sign**.

6.2. Add the plugin frameworks

Embed only the plugin frameworks whose hardware your app needs. For an app that supports NFC plus an external reader you would typically embed:

Plugin framework	Purpose
`plugins/NFC/iOSNfcPlugin.framework`	NFC reader session (built-in iPhone NFC)
`plugins/Feitian/Feitian.framework`	FTSafe iR301 / bR301 Bluetooth + Lightning smart card readers
`plugins/Tactivo/libTactivo.framework`	PreciseBiometrics Tactivo smart card reader
`plugins/Grabba/Grabba.framework`	Grabba external card reader
`plugins/Tech5/Tech5AirSnapFinger.framework` **plus 4 vendor frameworks**	Camera-based fingerprint capture (optional). See note below on Tech5 — five frameworks must be embedded together.

For each plugin framework you embed, set **Embed = Embed & Sign** in the same **Frameworks, Libraries, and Embedded Content** section as the toolkit framework.

> **Note: Tech5 — five frameworks, all Embed & Sign.** Unlike the other plugins, the Tech5 fingerprint plugin requires all of the following to be added to the target and individually set to **Embed & Sign**:
>
> - `plugins/Tech5/Tech5AirSnapFinger.framework` (the ICP plugin wrapper)

- `plugins/Tech5/AirSnapFingerUI.framework` (Tech5 vendor)
- `plugins/Tech5/T5AirSnapFramework.framework` (Tech5 vendor)
- `plugins/Tech5/ncnn.framework` (Tech5 vendor — ML runtime)
- `plugins/Tech5/opencv2.framework` (Tech5 vendor — image processing)

This is iOS's nested-framework code-signing requirement: Xcode's **Embed & Sign** only re-signs the top-level embedded framework with the integrator's certificate, leaving any nested children with the vendor's original signature, which iOS refuses to load at app launch (the `dyld[…]: Library not loaded: @rpath/AirSnapFingerUI.framework/… code signature invalid` symptom). Adding each of the five frameworks as a separate top-level embedded framework lets Xcode re-sign each one individually, which is what dyld and the system loader expect.

6.3. Link the static dependencies

If your code calls `xmlsec` or OpenSSL APIs directly, link the static archives shipped under `ext/`. The bundled samples link the following set:

Library	Source path
`libxmlsec.a`	`ext/xmlsec/lib/iOS/libxmlsec.a`
`libssl.a`, `libcrypto.a`	`ext/openssl/lib/iOS/`

Add these via **Target > General > Frameworks, Libraries, and Embedded Content > + > Add Other > Add Files**.

Note: Link order. If your app links `libxmlsec.a` directly, the link line must list `xmlsec` before `openssl` (xmlsec depends on OpenSSL symbols, not the other way around).

6.4. Configure build settings

Setting	Value
`IPHONEOS_DEPLOYMENT_TARGET`	13.0 or higher (sample uses 13.0)
`SWIFT_VERSION`	5.0 (for Swift targets)
`TARGETED_DEVICE_FAMILY`	`1,2` (iPhone + iPad)
`FRAMEWORK_SEARCH_PATHS`	Include the `lib/`, `plugins/NFC`, `plugins/Feitian`, `plugins/Tactivo`, `plugins/Grabba`, `plugins/Tech5` roots relative to your project
`HEADER_SEARCH_PATHS`	`ext/openssl/include/iOS`, `ext/xmlsec/include/MacOSX`
`LIBRARY_SEARCH_PATHS`	`ext/openssl/lib/iOS`, `ext/xmlsec/lib/iOS`
`ENABLE_BITCODE`	`NO` (Xcode 14+ default)

Adjust the path prefixes to match where you copied or referenced the SDK relative to your Xcode project.

6.5. Update Info.plist

Add the keys below to your application's `Info.plist`. The exact set depends on which reader plugins you embed.

```
<!-- NFC reader session: required to talk to Emirates ID over NFC. -->
<key>com.apple.developer.nfc.readersession.iso7816.select-identifiers</key>
<array>
    <string>A000000243001300000000010101</string>
    <string>A00000024300130000000103</string>
    <string>A00000024300130000000101</string>
    <string>A000000243001300000000010109</string>
    <string>A00000024300130000000001FF</string>
</array>
<key>com.apple.developer.nfc.readersession.formats</key>
<array>
    <string>TAG</string>
    <string>NDEF</string>
</array>

<!-- Background modes for Bluetooth / Lightning smart card readers. -->
<key>UIBackgroundModes</key>
<array>
    <string>bluetooth-central</string>
    <string>bluetooth-peripheral</string>
    <string>external-accessory</string>
</array>

<!-- External-accessory protocols for the readers you support. -->
<key>UISupportedExternalAccessoryProtocols</key>
<array>
    <string>com.precisebiometrics.sensor</string>
    <string>com.precisebiometrics.manager</string>
    <string>com.precisebiometrics.ccidbulk</string>
    <string>com.precisebiometrics.ccidinterrupt</string>
    <string>com.precisebiometrics.ccidcontrol</string>
    <string>com.ftsafe.bR301</string>
    <string>com.ftsafe.iR301</string>
    <string>com.ftsafe.br301</string>
    <string>com.ftsafe.ir301</string>
    <string>com.keyxentic.kxpos01</string>
</array>
```

6.6. Update entitlements

Your target's `.entitlements` file must declare the NFC capability. The `formats` array must match the one in `Info.plist` — include `TAG` (required for the Emirates ID APDU session, which uses `NFCTagReaderSession`) and `NDEF` (used by `NFCNDEFReaderSession`; required if any of your app flows also read NDEF-formatted tags, and harmless to include even if they don't):

```
<key>com.apple.developer.nfc.readersession.formats</key>
<array>
    <string>TAG</string>
    <string>NDEF</string>
</array>
```

Xcode adds this automatically when you enable **Near Field Communication Tag Reading** under **Signing & Capabilities** — verify both `TAG` and `NDEF` landed and match `Info.plist`.

6.7. Bundle configuration files

Add the `config_*` files for your environment to the Xcode project (check **Copy items if needed** so they end up inside the app bundle), then copy them to the application's Documents directory at startup (see Section 4.4 for the AppDelegate snippet).

6.8. Initialize the toolkit

Minimal Swift initialization once the configuration files are in the Documents directory:

```
import IDCardToolkit

// ...
do {
    let toolkit = try Toolkit()
    try toolkit.initialize()
    // ... call API methods (reader list, public data read, signing, etc.)
} catch let error as ToolkitException {
    print("Initialize failed: \(error)")
}
```

Refer to `ID_Card_Toolkit_Developer_Guide_ObjectiveC_Swift.pdf` in `doc/` for the full API surface and per-operation examples.

7. Code Signing

All embedded frameworks must carry a valid signature inside the `.app` bundle, otherwise iOS refuses to launch the app. For Xcode-managed signing, leave **Code Sign On Copy** enabled (the default) on every framework added to the **Embed Frameworks** build phase. Xcode then re-signs each embedded framework with your team identity at build time.

If you have a paid Developer account, your bundle will install on any device. With a free personal team, the provisioning profile expires every seven days and the app must be rebuilt and re-deployed from Xcode after expiry.

8. Troubleshooting

Symptom	Likely cause	Resolution
Library not loaded: `IDCardToolkit.framework`	Framework added but Embed setting is **Do Not Embed**.	Change the Embed setting to **Embed & Sign** under **Target > General > Frameworks**.
`dyld: Library not loaded — iOSNfcPlugin`	Plugin framework referenced in code but not added to the Embed Frameworks phase.	Embed the plugin in the same way as the toolkit framework.
Undefined symbols at link time (`_xmlSecInit`, `_SSL_CTX_new`, `_EVP_*`)	The static archives shipped under `ext/` are not on the link line.	Add `libxmlsec.a`, `libssl.a` and `libcrypto.a` from `ext/.../lib/iOS/`. Link xmlsec before openssl.
`Initialize()` returns error 189 — "Invalid or incomplete configuration data"	`config_*` files were not deployed alongside the app.	Confirm that `AppDelegate`'s `copyFilesToDocumentDirectory` ran (check the app's Documents directory after launch).
NFC reader session never starts	Missing entitlement or Info.plist key.	Verify the **Near Field Communication Tag Reading** capability is enabled and the `iso7816 select-identifiers` list is present in Info.plist.
Cannot use NFC on a paid Apple Developer account	Team is enrolled but the NFC capability is not provisioned for the App ID.	Enable the NFC capability for the App ID in the Apple Developer portal.
`pod install` fails on simulator builds only	Some pods (e.g. `SwiftyTesseract`) ship only device slices.	Build for a physical device, or update the Podfile to include simulator slices.
App crashes on launch — "code signature invalid"	Framework was not re-signed with your team identity during embed.	Make sure **Embed & Sign** (not **Embed Without Signing**) is selected for every plugin.

9. Reference Documents

The SDK ships the following PDFs under `doc/`:

- `ID_Card_Toolkit_Overview.pdf` — feature and device-support matrix
- `ID_Card_Toolkit_Developer_Guide_ObjectiveC_Swift.pdf` — full API reference
- `ID_Card_Toolkit_Build_Instructions_iOS_MAC_Sample_Project.pdf` — extended build notes
- `ID_Card_Toolkit_Authenticate_Face_iOS.pdf` — face-authentication flow (optional feature)
- `ID_Card_Toolkit_Programmers_Reference.pdf` — additional API detail
- `ID_Card_Toolkit_References.pdf` — supporting documents and standards